

QUEUE

Q6)

Question:

DS	Session	Queue	Question Information	Level 1	Challenge 56
Question description Lalitha is a B.Tech student. During her final year Campus Interview, she has an opportunity to get a job in a software company in Bangalore. The company provides Five months training period with Rs.30000/month Package. Then it will be incremented to Rs.55000 per month. At the end of the training, the examination was conducted for all freshers, Lalitha got a question paper and one of the questions comes under the concept of Queues in data structure that is Circular Linked List Implementation in Circular Queue. Can you help?					
The diagram illustrates the operations of a Circular Queue implemented using a Circular Linked List. It shows two rows of nodes, each with a 'Front' and 'Rear' pointer. The top row shows the initial state with Front and Rear pointers at node 14. Operations: enqueue(14), enqueue(22), enqueue(-6). The bottom row shows the state after dequeuing 14 and 22, and enqueuing 9 and 20. Operations: dequeue(), dequeue(), enqueue(9), enqueue(20).					
Constraints First 5 elements inserted in array by rear, 5 elements inserted into array, remaining elements inserted into array.					
Input Format First line indicates the number of elements to be inserted in the queues. Second line indicates the elements.					
Output Format first line indicates the dequeue element of Q2 Second line indicates the dequeue element of Q1 Third line line indicates the dequeue element of Q0					

PROGRAM:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node {
5      int data;
6      struct Node *next;
7  };
8
9  struct Node *front = NULL;
10 struct Node *rear = NULL;
11
12 void enqueue(int data) {
13     struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
14     newNode->data = data;
15
16     if (front == NULL) {
17         front = rear = newNode;
18         rear->next = front;
19     } else {
20         newNode->next = front;
21         rear->next = newNode;
22         rear = newNode;
23     }
24 }
25
26 int dequeue() {
27     int value;
28     struct Node *temp;
29
30     if (front == NULL)
31         return -1;
32
33     value = front->data;
34
35     if (front == rear) {
36         free(front);
37         front = rear = NULL;
38     } else {
39         temp = front;
40         front = front->next;
41         rear->next = front;
42         free(temp);
43     }
44
45     return value;
46 }
```

```
void display() {
    struct Node *temp;

    if (front == NULL)
        return;

    temp = front;

    do {
        printf("%d ", temp->data);
        temp = temp->next;
    } while (temp != front);
}

int main() {
    int n, x;

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        enqueue(x);
    }

    printf("Elements in Circular Queue are:");
    display();
    printf("\n");

    printf("Deleted value = %d\n", dequeue());

    printf("Deleted value = %d", dequeue());

    printf("Elements in Circular Queue are:");
    display();
    printf("\n");

    return 0;
}
```

OUTPUT:

TEST CASE 1:

```
10
1 2 3 4 5 6 7 8 9 10
Elements in Circular Queue are:1 2 3 4 5 6 7 8 9 10
Deleted value = 1
Deleted value = 2Elements in Circular Queue are:3 4 5 6 7 8 9 10
```

TEST CASE 2:

```
10
16 17 18 19 20 11 12 13 14 15
Elements in Circular Queue are:16 17 18 19 20 11 12 13 14 15
Deleted value = 16
Deleted value = 17Elements in Circular Queue are:18 19 20 11 12 13 14 15
```